

Tema: Proyecto web-scraping a la página planetadelibros.com

Materia: Interfaz de usuario

Profesor: Demetrio Meister

Grupo PsyNet

Integrantes:

Daniel Llanes

Constanza Rodriguez

Veronica Ramirez

Flavia Perín

Repositorio: <https://github.com/DanielNicolasLlanes/Scraper-PlanetaLibros.git>

```
import requests
```

```
from bs4 import BeautifulSoup
```

```
from selenium import webdriver
```

```
from selenium.webdriver.common.by import By
```

```
from selenium.webdriver.chrome.options import Options
```

```
from selenium.webdriver.chrome.service import Service
```

```
from webdriver_manager.chrome import ChromeDriverManager
```

```
from selenium.webdriver.support.ui import WebDriverWait
```

```
from selenium.webdriver.support import expected_conditions as EC
```

```
from selenium.common.exceptions import TimeoutException, NoSuchElementException,  
StaleElementReferenceException
```

```
from time import sleep
```

```
import random
```

```
# --- Configuración de Selenium ---
```

```
options = Options()
```

```
options.add_argument("start-maximized")
```

```
options.add_argument("--disable-blink-features=AutomationControlled")
```

```
options.add_argument("user-agent=Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36  
(KHTML, like Gecko) Chrome/120.0.0.0 Safari/537.36")
```

```
options.add_argument("--headless")
```

```
# Usar webdriver-manager para gestionar ChromeDriver automáticamente
```

```
service = Service(ChromeDriverManager().install())
```

```
driver = webdriver.Chrome(service=service, options=options)
```

```
# --- 1. Obtener URL de Categoría con Requests/BS4 ---
```

```
url = "https://www.planetadelibros.com/"
```

```
response = requests.get(url)
```

```
soup = BeautifulSoup(response.text, "html.parser")
```

```
tema = input("📖 Ingresá una temática (ej: terror, ficción, filosofía): ").strip().lower()
```

```
links = soup.find_all("a", class_="MenuPrincipalDesktop_nav__linkItem__G87vt")
categoria_url = None
```

```
for a in links:
```

```
    if tema in a.text.strip().lower():
```

```
        categoria_url = a["href"]
```

```
        if not categoria_url.startswith("http"):
```

```
            categoria_url = "https://www.planetadelibros.com" + categoria_url
```

```
        break
```

```
if not categoria_url:
```

```
    print("⚠️ No se encontró esa categoría.")
```

```
    driver.quit()
```

```
    exit()
```

```
print(f"✅ URL encontrada: {categoria_url}")
```

```
# --- 2. Navegar y Manejar Cookies con Selenium ---
```

```
driver.get(categoria_url)
```

```
# Intentar aceptar cookies
```

```
try:
```

```
    print("🕒 Buscando pop-up de cookies...")
```

```
    WebDriverWait(driver, 5).until(
```

```
        EC.presence_of_element_located((By.XPATH, "//button[contains(text(), 'Aceptar todas las cookies')]"))
```

```
    )
```

```
    cookie_button = WebDriverWait(driver, 5).until(
```

```
        EC.element_to_be_clickable((By.XPATH, "//button[contains(text(), 'Aceptar todas las cookies')]"))
```

```
    )
```

```
    cookie_button.click()
```

```
    print(f"✅ Cookies aceptadas.")
```

```
    sleep(1)
```

```
except TimeoutException:
```

```
    print("ℹ️ No se encontró o ya se cerró el pop-up de cookies.")
```

```
except Exception as e:
```

```
    print(f"⚠️ Error al intentar cerrar el pop-up de cookies: {e}")
```

```
try:
```

```
    WebDriverWait(driver, 10).until(
```

```

        EC.presence_of_all_elements_located((By.CSS_SELECTOR, ".Libro_libro__Yal5f"))
    )
except TimeoutException:
    print("⚠️ No se cargaron los libros a tiempo.")
    driver.quit()
    exit()

```

--- 3. Extraer enlaces y datos de la lista de libros ---

```

books_elements = driver.find_elements(By.CSS_SELECTOR, ".Libro_libro__Yal5f")
print(f"📖 Se encontraron {len(books_elements)} libros.")

```

```
books_data = []
```

Extraer los datos y enlaces de la lista de elementos EN LA PÁGINA DE CATEGORÍA
for book_element in books_elements[:10]:

try:

```
    link_element = book_element.find_element(By.CSS_SELECTOR, "a")
```

```
    title = link_element.get_attribute("title")
```

```
    author_element = book_element.find_element(By.CSS_SELECTOR,
"span[class*='LibroAutores_autorNombreTruncado']")
    author = author_element.get_attribute("title")
```

```
    link = book_element.find_element(By.CSS_SELECTOR, "a").get_attribute("href")
```

```
    books_data.append({
        'title': title,
        'author': author,
        'link': link,
    })
```

except NoSuchElementException as e:

```
    print(f"⚠️ Error al obtener datos básicos de un libro: {e}")
```

except StaleElementReferenceException:

```
    print("⚠️ Stale element al obtener datos básicos. Reintentando...")
    continue

```

--- 4. Iterar sobre los enlaces para obtener sinopsis y precio ---

```
print("\n🔍 Extrayendo detalles (Sinopsis/Precio)...")
```

for i, book in enumerate(books_data):

try:

```
    # Navegar a la página del libro
```

```
    driver.get(book['link'])
```

```
    sleep(random.uniform(1.5, 3.5)) # Pausa aleatoria para parecer más humano
```

--- Obtener Sinopsis ---

```
synopsis = "Sinopsis no disponible"
```

```

try:
    # 1. Buscar el contenedor principal de los párrafos de la sinopsis

    article = driver.find_element(By.CSS_SELECTOR, "div[class*='FichaLibro_sinopsis']")
    article")

    # 2. Encontrar todos los elementos <p> dentro del artículo
    paragraphs = article.find_elements(By.CSS_SELECTOR, "p")

    # 3. Concatenar el texto de todos los párrafos
    full_synopsis = []
    for p in paragraphs:
        # Usamos .text para obtener el contenido visible del párrafo
        text = p.text.strip()
        if text: # Solo añadir si el texto no está vacío
            full_synopsis.append(text)

    synopsis = "\n\n".join(full_synopsis)

except NoSuchElementException:
    synopsis = "Sinopsis no disponible (No se encontró el elemento principal)"
except Exception as e:
    synopsis = f"Error al extraer sinopsis: {type(e).__name__}: {e}"

# --- Obtener Precio ---
prices = {}
price = "Precio no disponible" # Valor por defecto
try:

    format_links = driver.find_elements(By.CSS_SELECTOR,
"a[class*='OpcionesCompra_btnFormato']")

    for link_price in format_links:
        try:
            format_text = link_price.find_element(By.CSS_SELECTOR,
"span[class*='OpcionesCompra_btnFormato__formato']").text
            price_text = link_price.find_element(By.CSS_SELECTOR,
"span[class*='OpcionesCompra_btnFormato__precio']").text

            format_text_lower = format_text.lower()

            if 'rústica' in format_text_lower or 'tapa blanda' in format_text_lower:
                prices['Físico'] = price_text
            elif 'ebook' in format_text_lower or 'epub' in format_text_lower:
                prices['eBook'] = price_text

        except NoSuchElementException:
            continue

```

```

price_output = []
if 'Físico' in prices:
    price_output.append(f"Físico: {prices['Físico']}")
if 'eBook' in prices:
    price_output.append(f"eBook: {prices['eBook']}")

if price_output:
    price = " | ".join(price_output)

except Exception as e:
    price = f"Error al extraer precio: {type(e).__name__}"

# --- Imprimir resultados ---
print(f"--- Libro {i + 1}/{len(books_data)} ---")
print(f"📖 Título: {book['title']}")
print(f"👤 Autor: {book['author']}")
print(f"$ Precio: {price}")
print(f"📄 Sinopsis: {synopsis}")
print(f"🔗 Enlace: {book['link']}")
print("-" * 60)

except Exception as e:
    print(f"⚠️ Error general al procesar el libro {book['link']}: {e}")
finally:
    driver.back()
    pass

# --- Finalizar ---
driver.save_screenshot("debug_final.png")
print("🖼️ Captura guardada: debug_final.png")

driver.quit()

```